

INTERNSHIP PROJECT REPORT TASK-2

Title

Develop and test custom correlation rules in a SIEM to detect mentioned attack techniques - “Credential Stuffing”, “DNS Tunneling” and “PowerShell Exploitation”.

1. Introduction

As part of the cybersecurity internship program, I was assigned the task of developing and testing custom correlation rules in a Security Information and Event Management (SIEM) platform. The objective was to understand how security events are collected, analyzed and correlated to identify malicious activities.

For this project, Elastic Security was used to create and test custom detection rules. The practical exercise focused on learning SIEM architecture, detection engineering, event monitoring and alert generation techniques used by modern Security Operations Centers (SOCs).

2. Background

Security Information and Event Management (SIEM) platforms help organizations collect and analyze logs from various systems. They provide centralized visibility and assist security analysts in identifying threats. Monitoring and analyzing this information manually can be difficult, which is why Security Information and Event Management (SIEM) solutions are widely used.

The attack techniques considered in this project include:

- Credential Stuffing • DNS Tunnelling • PowerShell Exploitation

Understanding these attack techniques is important because they are frequently associated with unauthorized access attempts, data theft, and malicious system activities.

3. Learning Objectives

The main aim of this project was to learn how a SIEM platform is used for security monitoring and threat detection. Through this project, I wanted to gain practical experience with Elastic Security and understand how security events can be monitored and analyzed.

The main objectives of this project were:

- To understand the basic working of a SIEM platform.
- To learn how to set up and use Elastic Security.
- To create custom detection rules for different attack scenarios.
- To generate and analyze sample security logs.
- To gain hands-on experience with log analysis and threat detection.

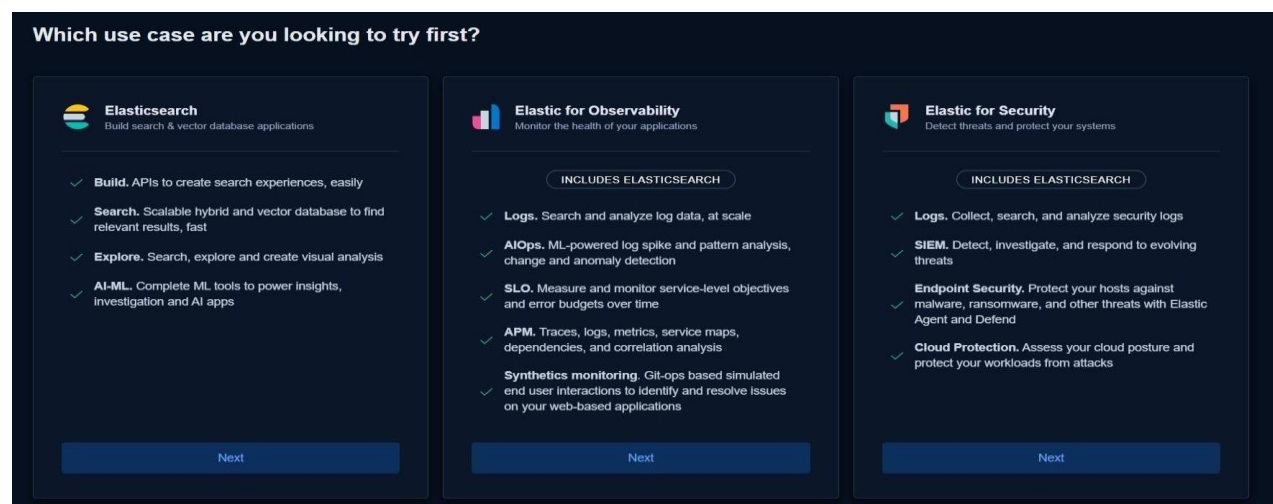
4. Activities and Tasks Performed

Step 1 - Creating Elastic Cloud Environment

I started the project by creating an Elastic Cloud account and selecting Elastic Security during the onboarding process. This helped me understand how SIEM platforms can be used for security monitoring and threat detection.

Step 2 - Elastic Security Use Case Selection

During the onboarding process, Elastic Cloud provided multiple use case options. For this project, I selected **Elastic Security** because it provides SIEM capabilities, security monitoring, threat detection, and alert management required for developing and testing custom detection rules.



Step 3 - Deployment Configuration

During the deployment setup process, different deployment options were available. The Elastic Cloud Hosted type was selected based on the project requirements to create the Elastic Security environment and proceed with the SIEM configuration.

Which deployment type would you like?

You can deploy another type later, if needed.

☐

**Elastic Cloud Serverless**

Fully managed, autoscaling servers — for operational simplicity.

- ✓ No server, node, or shard management required
- ✓ Automated scaling, upgrades, and cluster operation
- ✓ Pay only for compute, storage, and features used

☐

**Elastic Cloud Hosted**

Dedicated servers you manage — for maximum control.

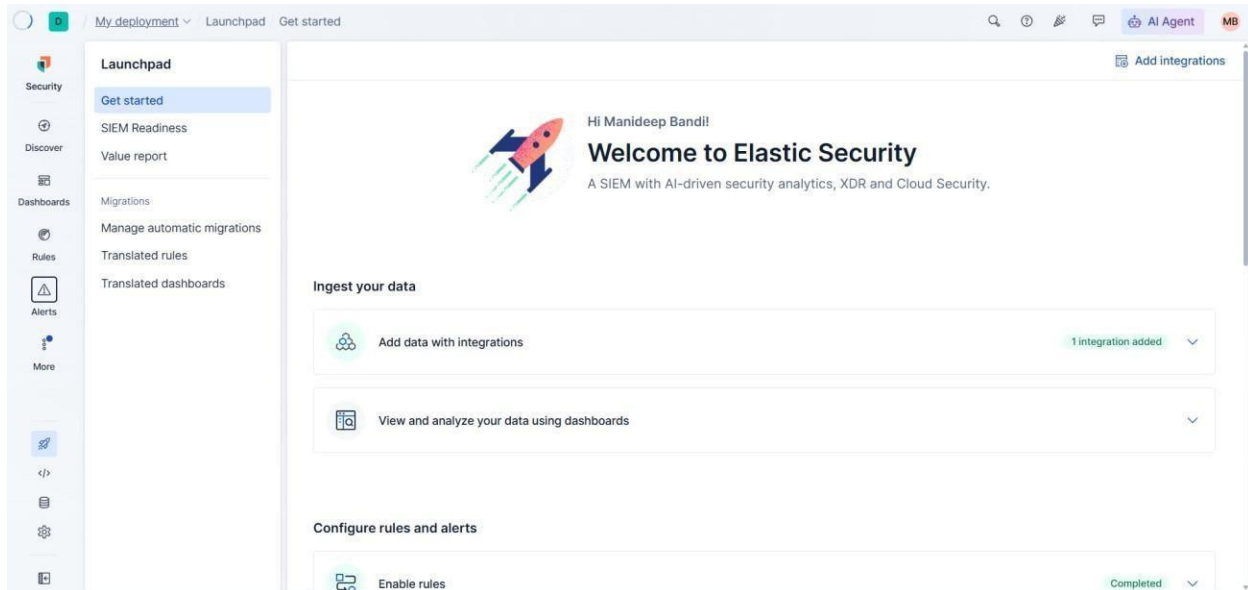
- ✓ Configure server, node, and shard resources
- ✓ Manual sizing and upgrades
- ✓ Pay flat rate for provisioned resources

Continue

[Learn more](#) 

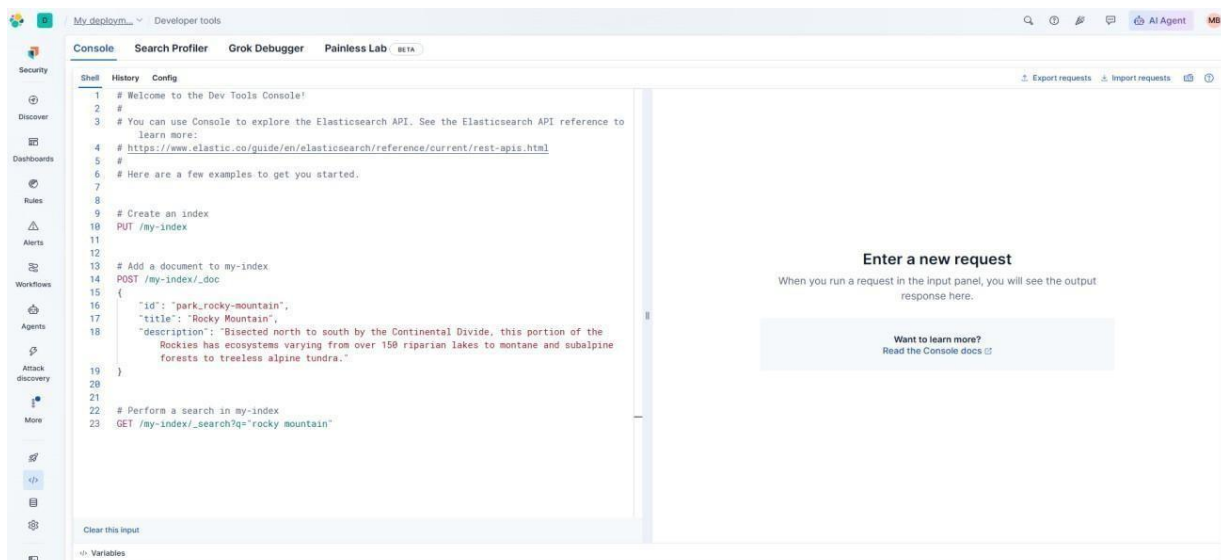
Step 4 - Accessing Elastic Security Dashboard

After completing the deployment setup, the Elastic Security dashboard was successfully accessed. This dashboard served as the main workspace for the project, where logs could be monitored, security rules could be created and alerts could be analyzed.



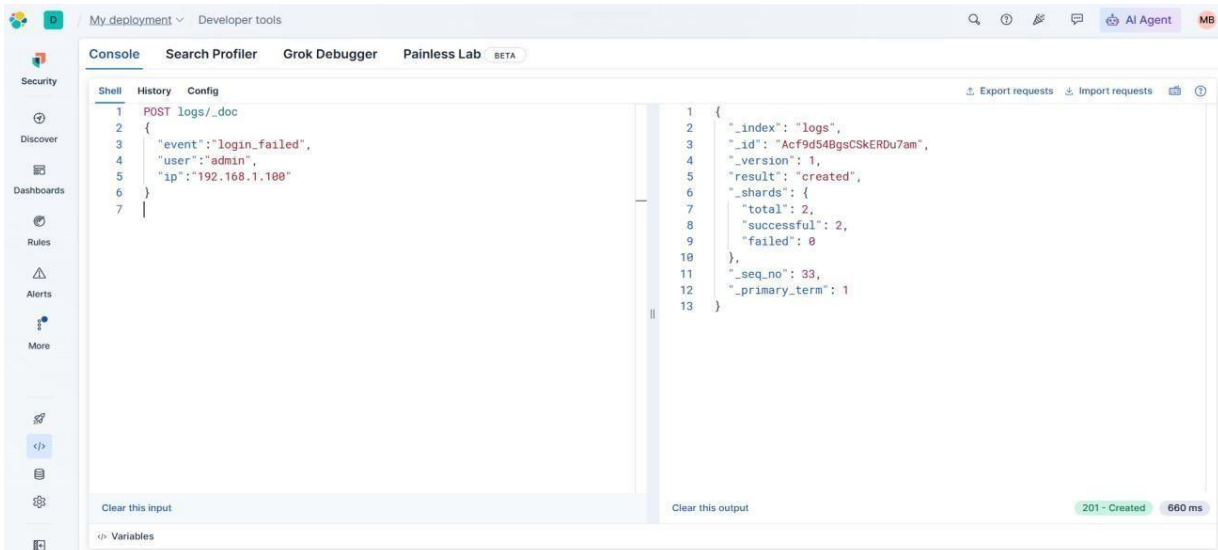
Step 5 - Accessing Elastic Dev Tools Console

After accessing the Elastic Security dashboard, I opened the Dev Tools Console. This console provides a built-in interface for interacting with Elasticsearch using API requests. It was later used to create custom indices, insert sample log data and verify the results required for testing SIEM detection rules.



Step 6 - Generating Test Logs for Credential Stuffing

To simulate a credential stuffing attack, I created sample failed login events using the Elastic Dev Tools Console. The logs contained multiple unsuccessful login attempts, representing a scenario where attackers try different username and password combinations to gain unauthorized access. After executing the request, the logs were successfully stored in the custom index, as confirmed by the successful response message. This test data was later used to validate the Credential Stuffing Detection Rule.



The screenshot displays the Elastic Dev Tools Console interface. On the left sidebar, the 'Security' section is expanded, showing options like Discover, Dashboards, Rules, Alerts, and More. The main console area is divided into two panes. The left pane, labeled 'Shell', shows a REST client request: a POST to 'logs/_doc' with a JSON body containing 'event': 'login_failed', 'user': 'admin', and 'ip': '192.168.1.100'. The right pane shows the response, which is a 201 status code indicating the document was created. The response body is a JSON object with fields for '_index' (logs), '_id' (Acf9d94BgsCSKERDu7am), '_version' (1), '_result' (created), '_shards' (total: 2, successful: 2, failed: 0), '_seq_no' (33), and '_primary_term' (1). At the bottom right, a status bar shows '201 - Created' and '660 ms'.

```
1 POST logs/_doc
2 {
3   "event": "login_failed",
4   "user": "admin",
5   "ip": "192.168.1.100"
6 }
7
```

```
1 {
2   "_index": "logs",
3   "_id": "Acf9d94BgsCSKERDu7am",
4   "_version": 1,
5   "_result": "created",
6   "_shards": {
7     "total": 2,
8     "successful": 2,
9     "failed": 0
10  },
11   "_seq_no": 33,
12   "_primary_term": 1
13 }
```

201 - Created 660 ms

Step 7 - Creating Credential Stuffing Detection Rule

I configured the custom Credential Stuffing Detection Rule by assigning the rule name "Credential Stuffing Detection" and adding a description explaining its purpose. I set the severity level to **Medium** and assigned a risk score of **50** to indicate the potential impact of repeated failed login attempts. I configured the rule to monitor logs containing the event "login_failed" and enabled it to run automatically at regular intervals. This helped me understand how SIEM rules are customized to detect specific attack techniques and generate security alerts.

The screenshot displays the Splunk SIEM interface for configuring a rule named "Credential Stuffing Detection". The left sidebar shows the navigation menu with options like Security, Management, Discover, Benchmarks, Shared exception lists, Dashboards, Rules, Alerts, Workflows, Agents, Attack discovery, and More. The main content area is divided into sections: About, Definition, and Schedule.

About

- Description:** Detects multiple failed login attempts indicating credential stuffing attack activity.
- Severity:** Medium
- Risk score:** 50
- Max alerts per run:** 100

Definition

- Index patterns:** apm-* transaction* auditbeat-* endgame-* filebeat-* logs-* packetbeat-* traces-apm* winlogbeat-* *elastic-cloud-logs*
- Custom query:** event: "login_failed"
- Custom query language:** KQL
- Rule type:** Query
- Timeline template:** None

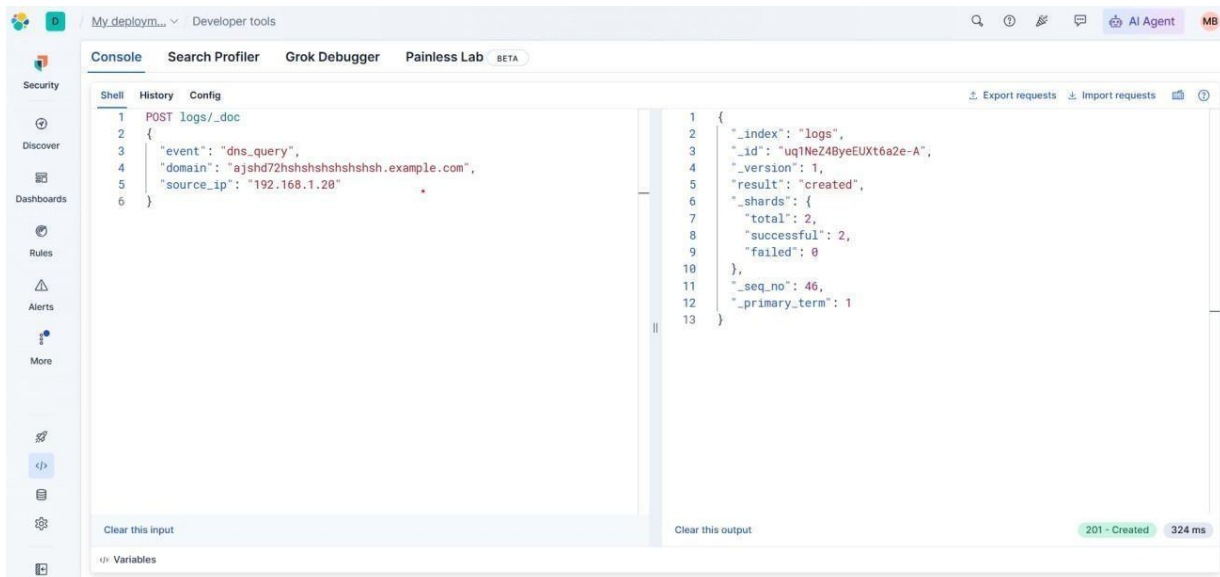
Schedule

- Runs every:** 1m
- Additional look-back time:** 5m

At the bottom, there are tabs for "Untitled Timeline" and "Unsaved".

Step 8 - Generating Test Logs for DNS Tunneling

To simulate DNS tunnelling activity, I created a sample DNS query log using the Elastic Dev Tools Console. The log contained a DNS request with an unusually long domain name and a source IP address, which resembles the type of traffic often associated with DNS tunnelling attacks. After executing the request, the log was successfully stored in the custom index, as indicated by the successful response message. This test data was later used to validate the DNS Tunneling Detection Rule.



Step 9 - Creating DNS Tunneling Detection Rule

I configured the custom DNS Tunnelling Detection Rule by assigning the rule name "DNS Tunnelling Detection" and adding a description explaining its purpose. I set the severity level to **Medium** and assigned a risk score of **50** to indicate the potential security risk associated with suspicious DNS query activity. I configured the rule to monitor logs containing the event "dns_query" and enabled it to run automatically at regular intervals. This helped me understand how SIEM detection rules can be configured to identify unusual DNS traffic patterns that may indicate DNS tunnelling attempts and generate alerts for further investigation.

The screenshot displays the Splunk SIEM interface for configuring a detection rule. The left sidebar shows navigation options like Security, Rules, Alerts, and Workflows. The main panel is titled 'DNS Tunnelling Detection' and includes tabs for Overview, Alerts, Rule exceptions, and Execution results. The 'About' section provides a description of the rule's purpose: detecting suspicious DNS query activity that may indicate DNS tunnelling. The 'Definition' section shows the rule's configuration, including index patterns, custom query, and schedule.

Rules

Management

Detection rules (SIEM)

Benchmarks

Shared exception lists

Discover

MITRE ATT&CK® Coverage

Rules

Alerts

Workflows

Agents

Attack discovery

More

ML job settings Add integrations Data view Security solution default

DNS Tunnelling Detection

Created by: 944974502 on May 30, 2026 @ 14:19:37.766 Updated by: 944974502 on May 30, 2026 @ 14:19:37.766

Last response: — Notify when alerts generated

Enable Add to chat Edit rule settings

About

Description

This rule detects suspicious DNS query activity that may indicate DNS tunnelling. DNS tunnelling is a technique used by attackers to hide data transfer or command-and-control communication within DNS requests.

Severity Medium

Risk score 50

Max alerts per run 100

Definition

Index patterns

apm-* transaction* auditbeat-* endgame-* filebeat-* logs-* packetbeat-* traces-apm-* winlogbeat-* elastic-cloud-logs*

Custom query event:"dns_query"

Custom query language KQL

Rule type Query

Timeline template None

Schedule

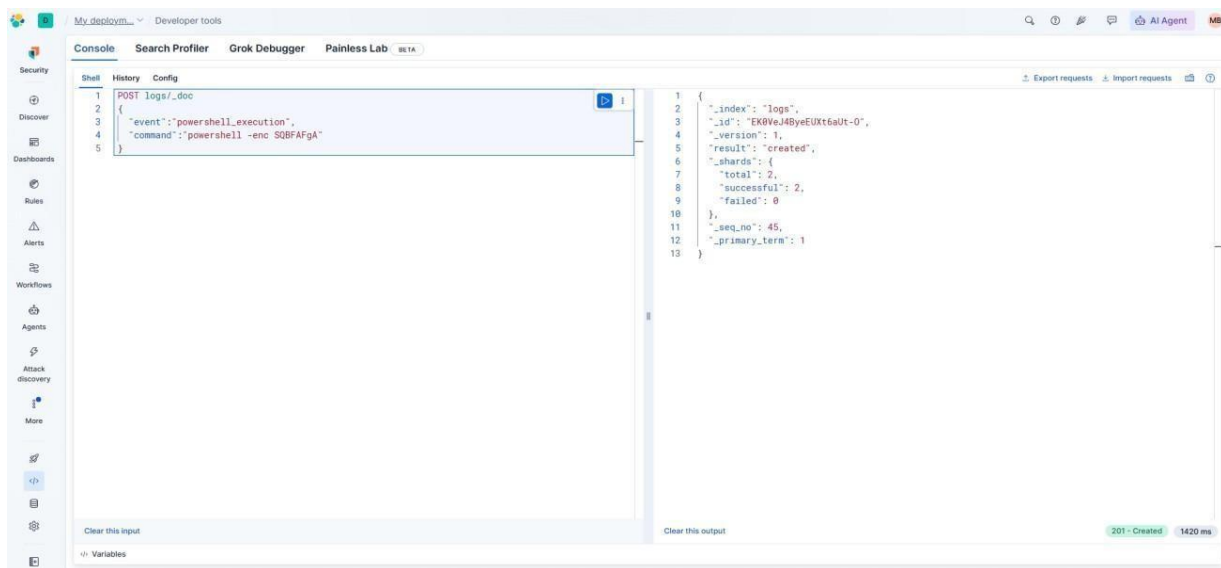
Runs every 1m

Additional look-back time 5m

Untitled Timeline Unsaved

Step 10 - Generating Test Logs for PowerShell Exploitation

To simulate a PowerShell exploitation attack, I created a sample PowerShell execution event using the Elastic Dev Tools Console. The log contained a PowerShell command associated with potentially suspicious script execution activity, representing a common technique used by attackers to execute malicious commands on a system. After executing the request, the log was successfully indexed and stored in Elasticsearch. This test data was later used to validate the PowerShell Exploitation Detection Rule.



Step 11 - Creating PowerShell Exploitation Detection Rule

I configured the custom PowerShell Exploitation Detection Rule by assigning the rule name **"PowerShell Exploitation Detection"** and adding a description explaining its purpose. I set the severity level to **Medium** and assigned a risk score of **50** to indicate the potential security impact of suspicious PowerShell activity. I configured the rule to monitor logs containing the event **"powershell_execution"** and enabled it to run automatically at regular intervals. This helped me understand how SIEM detection rules can be customized to identify potentially malicious PowerShell execution activities and generate security alerts for further investigation.

The screenshot displays the Splunk SIEM interface for configuring a detection rule. The left sidebar shows the navigation menu with options like Security, Discover, Dashboards, Rules, Alerts, Workflows, Agents, Attack discovery, and More. The main content area is titled 'PowerShell Exploitation Detection' and includes tabs for Overview, Alerts, Rule exceptions, and Execution results. The 'Overview' tab is active, showing the rule's description, severity, risk score, and max alerts per run. The 'Definition' section on the right lists index patterns, custom query, custom query language, rule type, and timeline template. The 'Schedule' section at the bottom shows the rule runs every 1m with an additional look-back time of 4m.

PowerShell Exploitation Detection

Created by: 944974502 on May 30, 2026 @ 14:24:25:188 Updated by: 944974502 on May 30, 2026 @ 14:24:25:188
Last response: succeeded at May 30, 2026 @ 14:24:26:904 Notify when alerts generated

About

Description
This rule detects potentially malicious PowerShell execution activity. Attackers often use PowerShell to execute encoded commands, download payloads, or perform post-exploitation actions. The rule monitors PowerShell-related events to identify suspicious command execution.

Severity Medium
Risk score 50
Max alerts per run 100

Definition

Index patterns
apm-* transaction* auditbeat-* endgame-* filebeat-* logs-* packetbeat-* traces-apm* winlogbeat-* *-elastic-cloud-logs*

Custom query event:"powershell_execution"
Custom query language KQL
Rule type Query
Timeline template None

Schedule

Runs every 1m
Additional look-back time 4m

Step 12 - Rule Execution and Monitoring

After creating and enabling all three custom SIEM detection rules, I monitored their execution through the Rule Monitoring dashboard in Elastic Security. The dashboard showed that the **Credential Stuffing Detection**, **DNS Tunnelling Detection** and **PowerShell Exploitation Detection** rules were running successfully without any errors or warnings. I verified the execution status, query processing time, and rule response for each rule. This step helped me understand how SIEM platforms continuously execute detection rules against incoming log data and ensure that security monitoring is functioning correctly.

Detection rules (SIEM)

[Settings](#) [Add Elastic rules 1850](#) [Manage value lists](#) [Import rules](#) [Create a rule](#)

[Installed Rules 3](#) [Rule Monitoring 3](#)

Rule Monitoring Overview

Auto gap fill status: Off

Last response summary

3 Rules

Status	Rules
Succeeded	3
Warning	0
Failed	0
No response	0

Rule gap summary

0 Total duration

Status	Total duration
Filled	0
In progress	0
Unfilled	0
Error	0

Tags

Gap fill status

Last response

Elastic rules (0)

Custom rules (3)

Enabled rules

Disabled rules

Showing 1-3 of 3 rules

Selected 0 rules

Select all 3 rules

Bulk actions

Refresh

Updated 17 seconds ago

On

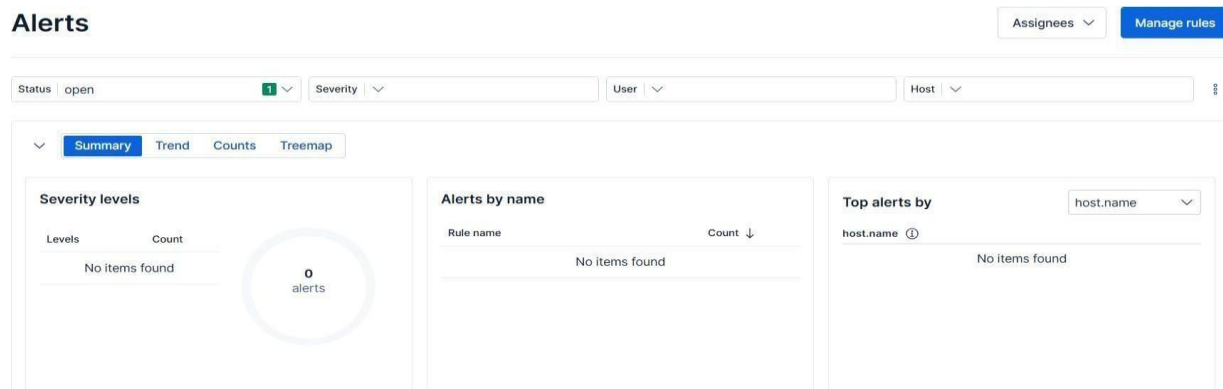
☐	Rule	Indexing Time (ms)	Query Time (ms)	Gap fill status	Last Gap (if any)	Unfilled gaps duration	Last response	Last run	Enabled	
☐	PowerShell Exploitation Detection	—	2	—	—	—	Succeeded	12 seconds ago	☒	⋮
☐	DNS Tunnelling Detection	—	12	—	—	—	Succeeded	15 seconds ago	☒	⋮
☐	Credential Stuffing Detection	—	4	—	—	—	Succeeded	9 seconds ago	☒	⋮

Rows per page: 20

1

Step 13 - Alert Validation

After creating and monitoring the detection rules, I verified that all three rules were enabled and functioned correctly. The Rule Monitoring dashboard confirmed successful execution of each rule. This verification process helped ensure that the SIEM environment was properly configured to detect the simulated attack activities.



After monitoring the detection rules, I verified that all three rules were enabled and executed successfully. The Rule Monitoring dashboard confirmed successful rule execution without errors. Although no alerts were generated during testing, the rules were configured correctly and were ready to generate alerts whenever matching events were detected.

5. Skills and Competencies Gained

- Developed a practical understanding of Security Information and Event Management (SIEM) concepts and their role in cybersecurity operations.
- Learned how to create and configure custom detection rules for identifying different attack techniques.
- Improved knowledge of Elasticsearch queries and their use in filtering and analyzing security events.
- Enhanced log analysis skills by creating, reviewing and validating test security logs.
- Learned how security events are monitored and how detection rules are executed within a SIEM environment.
- Developed the ability to identify suspicious activities related to credential stuffing, DNS tunnelling and PowerShell exploitation.

6. Feedback and Evidence

Feedback

This project provided me with practical experience in working with a Security Information and Event Management (SIEM) platform using Elastic Security. Through the process of creating custom detection rules, generating test logs and monitoring rule execution, I gained a better understanding of how security events are collected, analyzed, and used to identify suspicious activities. The hands-on implementation of Credential Stuffing, DNS Tunnelling, and PowerShell Exploitation detection rules helped me improve my knowledge of security monitoring, log analysis, and threat detection.

Evidence Collected

- Elastic Security deployment and dashboard configuration screenshots.
- Elastic Dev Tools Console screenshots used for generating test logs.
- Sample Credential Stuffing log events created for testing.
- Sample DNS Tunnelling log events created for testing.
- Sample PowerShell Exploitation log events created for testing.
- Credential Stuffing Detection Rule configuration and overview screenshots.
- DNS Tunnelling Detection Rule configuration and overview screenshots.
- PowerShell Exploitation Detection Rule configuration and overview screenshots.
- Rule Monitoring dashboard screenshots showing successful execution of all custom rules.
- Screenshots demonstrating rule validation and monitoring activities performed during the project.

7. Challenges and Solutions

During the project, I encountered several challenges while working with the Elastic Security platform for the first time. Since I was new to SIEM technologies, understanding the overall workflow and navigation of the platform required additional time and effort. To overcome this, I explored each section of the dashboard individually and learned the purpose of different modules such as Rules, Alerts, Dashboards and Developer Tools.

Generating and testing security events was also challenging because there were no real log sources connected to the environment. To solve this problem, I used the Elastic Dev Tools Console to manually create sample logs that simulated Credential Stuffing, DNS Tunnelling, and PowerShell Exploitation activities. These logs allowed me to validate the custom detection rules and understand how security events are processed within a SIEM platform.

Despite these challenges, each issue provided an opportunity to learn more about SIEM operations, detection rule development and log analysis. Working through these problems independently improved my troubleshooting skills and strengthened my understanding of security monitoring concepts.

8. Outcomes and Impact

This project provided me with valuable hands-on experience in working with a Security Information and Event Management (SIEM) platform. Before starting the project, I had only a basic understanding of how SIEM solutions are used in cybersecurity. Through the practical activities performed during this project, I gained a clearer understanding of how security events are collected, stored, monitored and analyzed within a centralized security platform.

By setting up the Elastic Security environment, I learned how cloud-based SIEM solutions can be deployed and used for security monitoring. Creating test logs for Credential Stuffing, DNS Tunnelling, and PowerShell Exploitation helped me understand how different attack techniques can be represented as security events and how these events can be used for detection and analysis.

The outcome of this project was the improvement of my analytical and problem-solving skills. During the implementation process, I encountered several configuration and testing challenges, which required me to explore different features of the platform, verify log ingestion, review rule settings and troubleshoot issues independently.

9. Conclusion

This project provided me with an opportunity to gain practical experience in working with a SIEM platform and understanding its role in cybersecurity. Throughout the project, I successfully set up an Elastic Security environment, generated test security events, and developed custom detection rules for Credential Stuffing, DNS Tunnelling and PowerShell Exploitation attack scenarios. These activities helped me understand how security events are monitored and how detection rules can be used to identify potentially malicious activities.

Overall, the project was a valuable learning experience that enhanced my understanding of SIEM technologies, log analysis, detection engineering and security operations. The knowledge and skills gained through this project will support my future learning and professional growth in the field of cybersecurity.